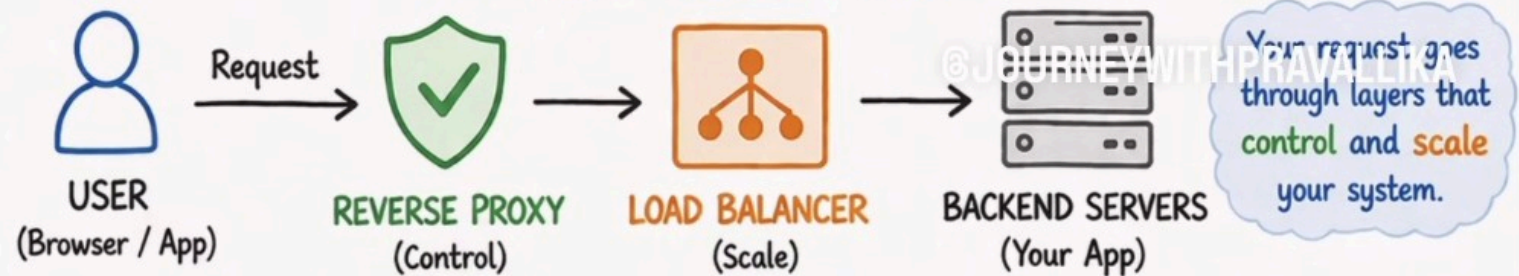


LET'S UNDERSTAND HOW YOUR REQUEST TRAVELS

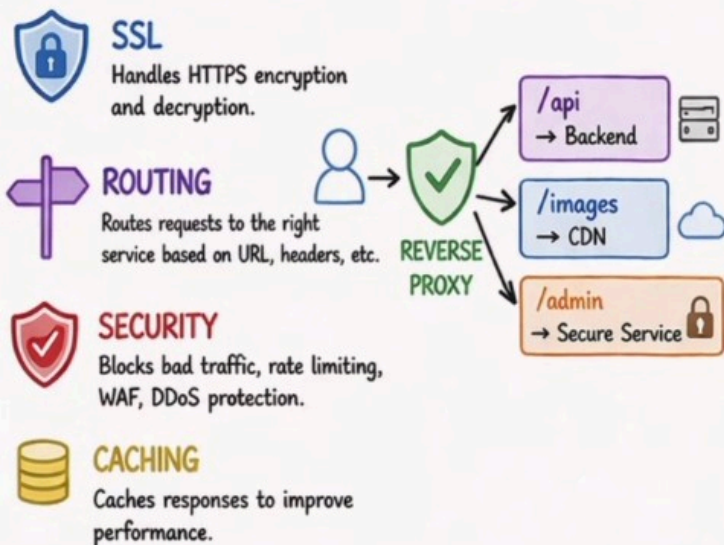
REAL-WORLD FLOW



User → Reverse Proxy (control) → Load Balancer (scale) → Backend Servers

REVERSE PROXY (GATEKEEPER)

Sits in front of your servers. Hides your backend and handles traffic smartly.
Users only see the proxy, not your servers.

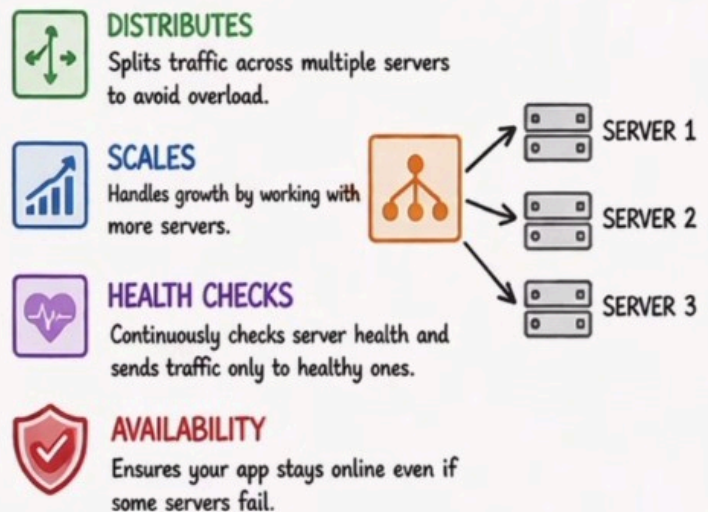


EXAMPLES:



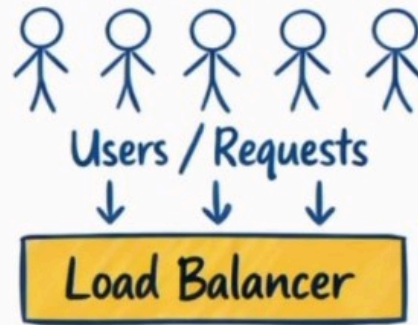
LOAD BALANCER (DISTRIBUTOR)

Sits between proxy and servers. Distributes traffic, scales your system, and keeps it highly available.

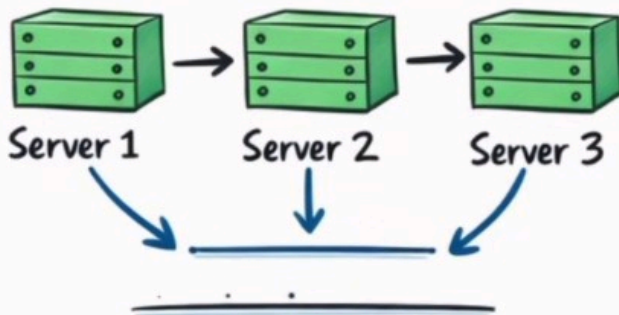


EXAMPLES:



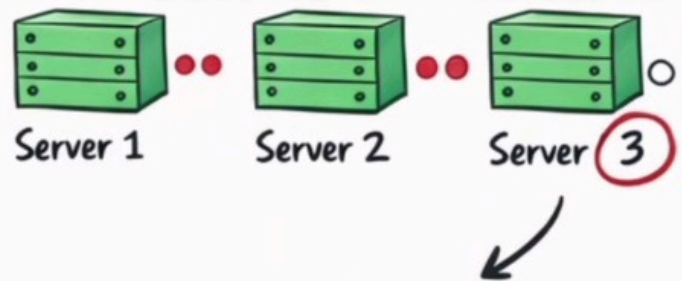


Round Robin



- 1 → 2 → 3 → repeat
- Equal but doesn't check load

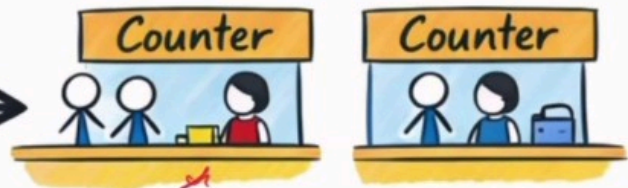
Least Connections



- Goes to least busy server



Shortest queue ↗



Simple

Smarter

HTTP vs HTTPS

What's the difference?

Both are ways your browser talks to a server,
but there's one **BIG** difference!

HTTP (Not Secure)

- Data is sent as plain text.
- Anyone in between can read it 😞



HTTPS (Secure)

- Data is encrypted.
- Even if someone intercepts it, they can't understand it. 🗝️



SSL/TLS in one line:

SSL/TLS encrypts your data so no one in between can read it. 🗝️

WHAT HAPPENS IN HTTPS?



① Browser says "Hello" to server

② Server sends its SSL certificate (proves it's real)

③ Browser & server agree on secret keys (handshake)

④ Now data is encrypted and sent safely!



QUICK RECAP

HTTP = open message 📧

HTTPS = locked message 🗝️

SDLC vs STLC – Simple Comparison

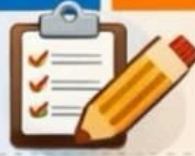
— For Software QA —

SDLC

STLC

1. Requirement Phase

✓ Gather Business Requirements



Identify Testable Requirements

2. Design Phase

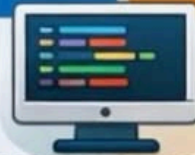
✓ System Design & Architecture



Test Strategy & Scenarios

3. Development Phase

✓ Develop the Code



Create Test Cases

4. Testing Phase

✓ Product Testing



Execute & Log Defects

5. Deployment

✓ Release to Production



Smoke Testing & Validation

6. Maintenance

✓ Bug Fixes & Updates



Regression Testing

SDLC = How Software is Built

STLC = How Software is Tested

TYPES OF AUTHENTICATION

BASIC AUTH

Username & Password
(Base64)

SESSION-BASED

Server stores
session ID

TOKEN-BASED

Token sent in headers

JWT

Stateless secure token

OAUTH

Third-party login access

API KEY

Unique key per client

MFA

Multiple verification steps

BIOMETRIC

Fingerprint / Face ID

MFA

Multiple verification steps

BIOMETRIC

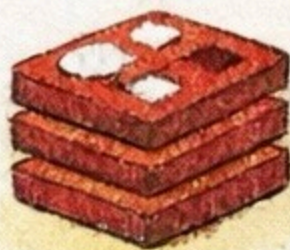
Fingerprint / Face ID

Follow for daily developer content



What is Redis?

- Redis = Remote Dictionary Server
- In-Memory Data Store for:
 - Cache
 - Database
 - Message Broker



Think: "Super-fast Key-Value Store in **RAM!**"

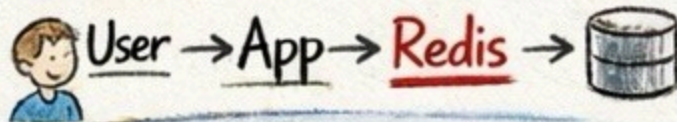
Why Redis?

- DBs are Slow for Frequent Reads!

Problem:

- High Latency!
- DB Load!
- Scalability Issues!

Solution: Use Redis as Cache!



⚡ Sub-ms Latency

⚡ Millions Ops/Sec!

When to Use Redis?

- 🏠 Caching (API Response)
- 🍪 Session Storage
- 📦 Rate Limiting
- 📊 Real-Time Analytics
- 🔗 Pub/Sub Messaging
- 🔒 Distributed Locks

Why is Redis So Fast?

• In-Memory:

- Data in RAM!
- 100x Faster than SSD!

• Single-Threaded

- No Locks!
- No Context Switches!
- One Event Loop!



User → App → Redis → DB → 

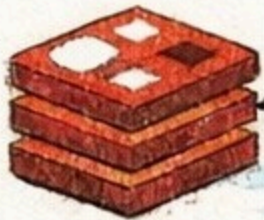
⚡ Sub-ms Latency ⚡ Millions Ops/Sec!

How Redis Works?

Client → TCP → Redis Server → In-Memory Data

Commands:

GET, SET, etc.



All in RAM!



All in RAM:



Hash Tables



Skip Lists

• Replication & Clustering!